DEEP LEARNING | SEQUENCE MODELS

# Attention Mechanisms

Bahdanau • Luong • Self-Attention • Cross-Attention • Multi-Head Attention

---

IIT Bhilai | Detailed classroom lecture | Continuation of Encoder–Decoder and Attention

# Learning objectives

Understand the computation, not just the names.

- Derive scores, softmax weights and weighted context vectors.
- Contrast additive (Bahdanau) with multiplicative (Luong) scoring.
- Explain self-attention and cross-attention with Q, K, V.
- Compute scaled dot-product and multi-head attention.
- Identify causal masking, tensor dimensions and practical limitations.

# Roadmap: five names, three different questions

### How is similarity computed?

Bahdanau: learned additive scorer
Luong: dot, general or concat scorer

### Where does information come from?

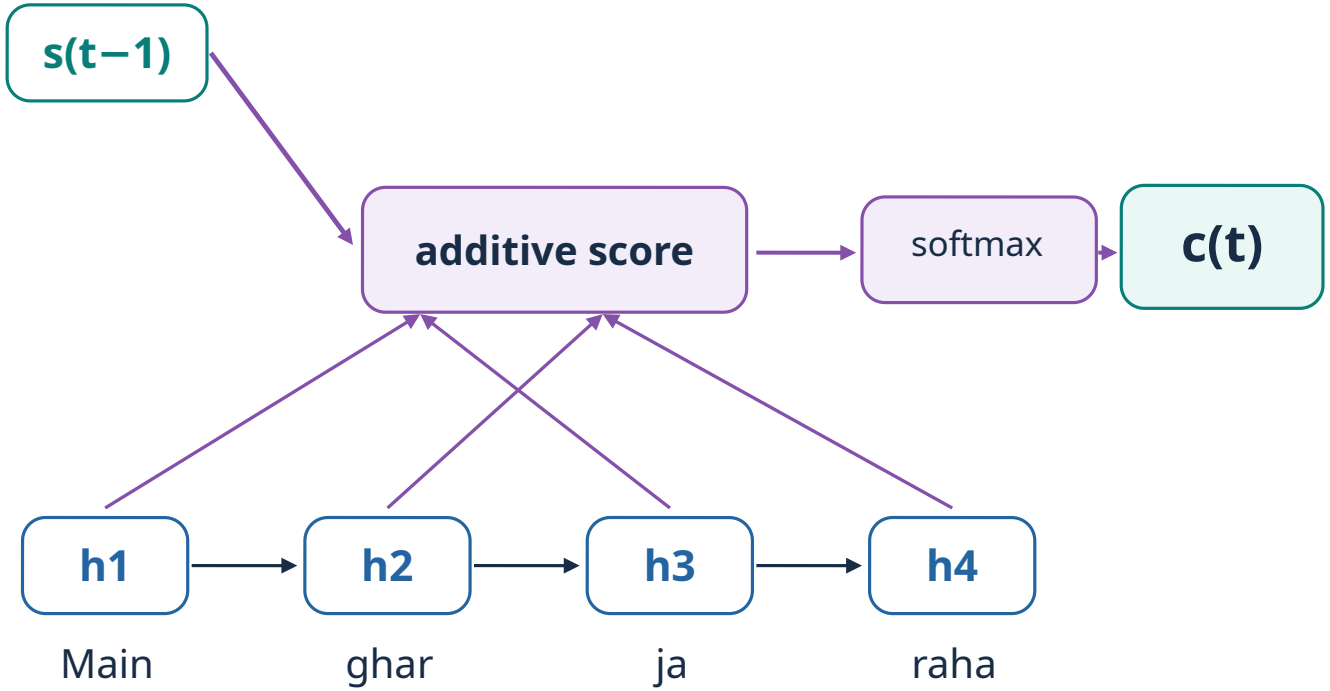
Self: Q/K/V from same sequence
Cross: Q from one, K/V from another

### How many retrieval patterns?

Multi-head: parallel learned projection spaces with separate attention patterns

$$\text{Attention}(Q,K,V) = \text{softmax}(\text{score}(Q,K)) \times V$$

# Recall: the bottleneck that motivated attention




- Earlier Seq2Seq compressed all source words into one fixed vector.
- Attention retains encoder states $h_1,\ldots,h_t$ and computes a new context for each target step.
- Different output words can attend to different source positions.

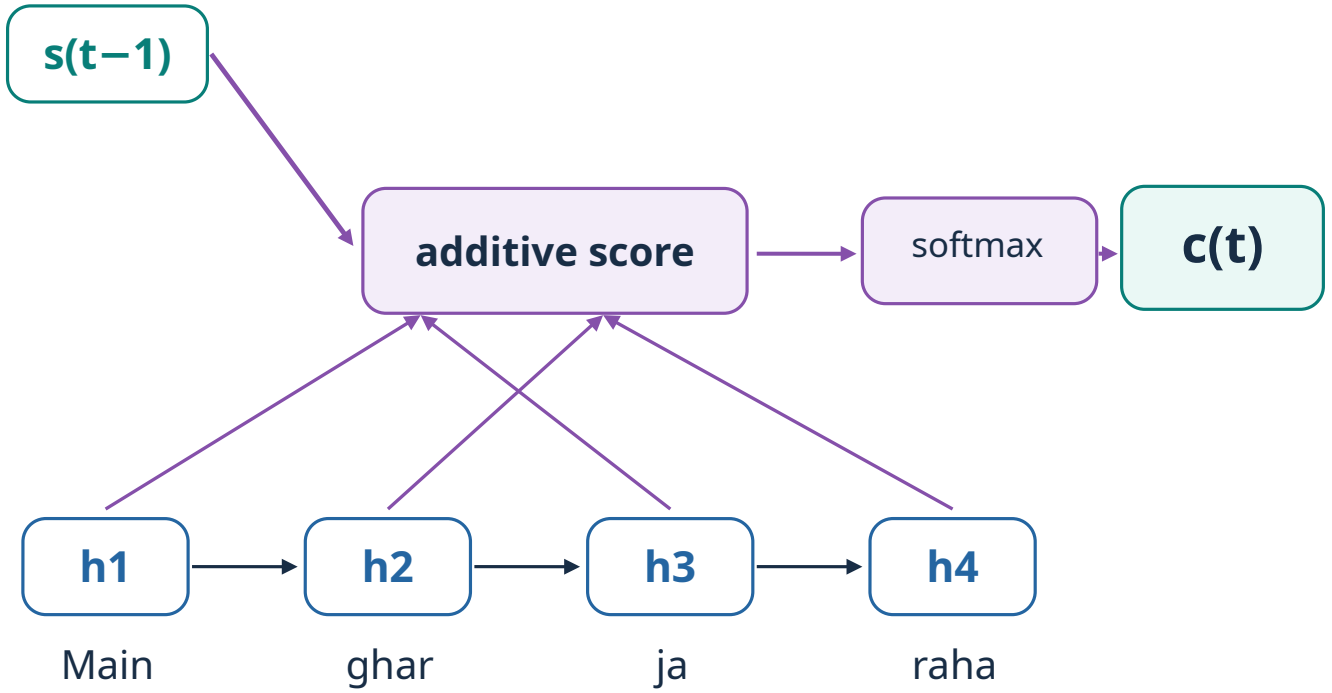
PART I

# Bahdanau Attention

Additive scoring in recurrent neural machine translation (2014 / ICLR 2015).

# Bahdanau: original recurrent encoder–decoder architecture

Decoder state and all encoder annotations jointly determine each alignment.




- Query: previous decoder state $s(t-1)$.
- Keys / values: encoder annotations $h_i$.
- Scorer: learned feed-forward network.
- Result: context $c_t$ supports the next target prediction.

# Why additive scoring?

The scorer learns a nonlinear compatibility between decoder history and each source annotation.

$$e_{t,i} = v_a^T \tanh(W_a s_{t-1} + U_a h_i + b_a)$$

**$s_{t-1}$**

What the decoder has generated so far (query).

**$h_i$**

Information at encoder position i (key and value).

**$v_a$, $W_a$, $U_a$**

Parameters learned jointly through training.

**The scalar $e_{t,i}$ is an unnormalized score — not a probability.**

# Bahdanau: from scores to attention weights

$$\alpha_{t,i} = \exp(e_{t,i}) / \Sigma_j \exp(e_{t,j})$$

$$c_t = \Sigma_i \alpha_{t,i} h_i$$

| Source token | Score $e_{t,i}$ | Softmax $\alpha_{t,i}$ |
|---|---|---|
| Main | 0 | 0.093 |
| ghar | 2 | 0.687 |
| ja | 0 | 0.093 |
| raha | 0 | 0.093 |
| hoon | −1 | 0.034 |

Rounded illustrative values; weights sum to approximately 1.

# Bahdanau: complete decoding step




$$s_t = \text{RNNdecoder}(s_{t-1}, E[y_{t-1}], c_t)$$

Exact decoder order and output layer can differ across implementations.

# Bahdanau worked example: weighted vector retrieval

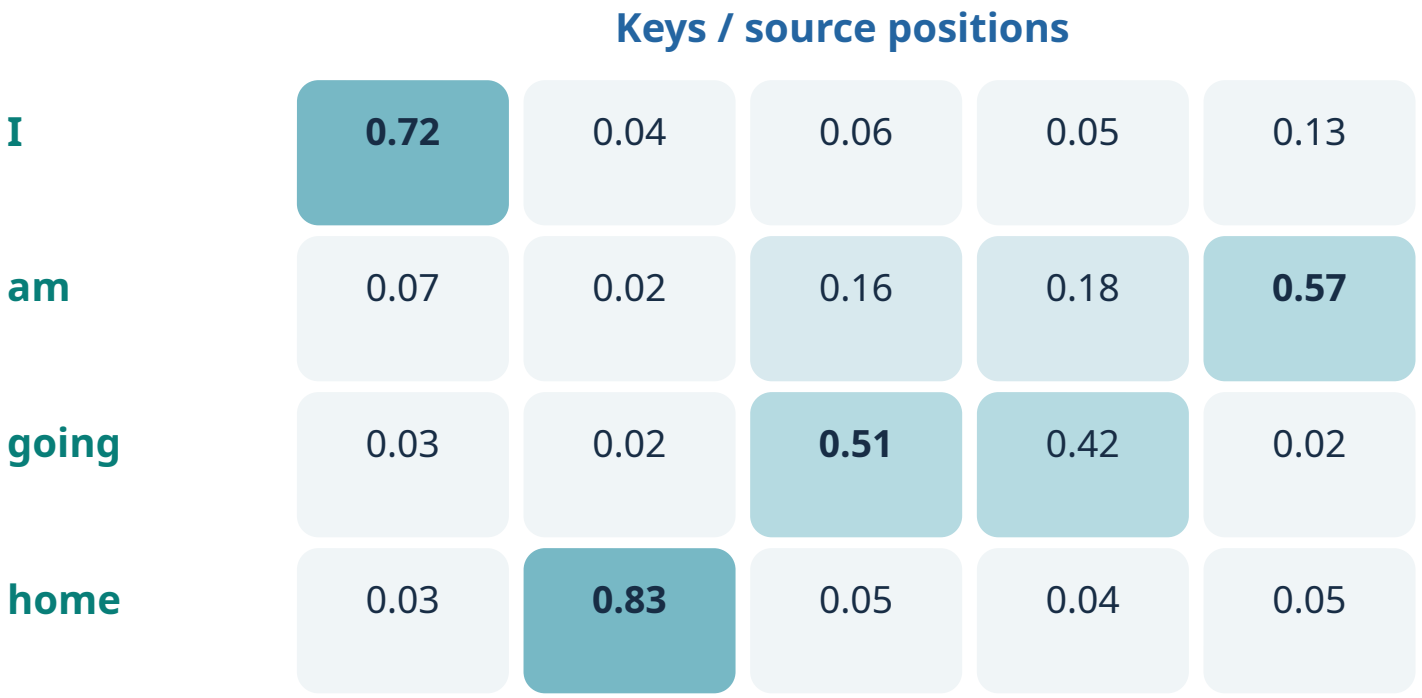
$$h_1 = [1,0] \quad h_2 = [0,1] \quad h_3 = [2,2]$$

$$e = [0, \ln 2, \ln 3] \quad \rightarrow \quad \alpha = [1/6, 2/6, 3/6]$$

$$c = (1/6)h_1 + (2/6)h_2 + (3/6)h_3 = [7/6, 4/3]$$

**Ask: Why is $h_3$ influential even though no single position is chosen?**

# Bahdanau: token-level alignment is soft




- One output may use several input tokens.
- The attention distribution changes when the decoder state changes.
- Weights support analysis but are not guaranteed faithful explanations.

Illustrative classroom heatmap; not measured weights from a trained network.

# Luong Attention: Basic Idea

Luong et al. (2015) | Multiplicative attention for encoder-decoder models

**Definition**

Luong attention scores each encoder hidden state against the current decoder state to determine which source information is most relevant for predicting the next word.

**Decoder state** $s_t$ → **Alignment scores** $e_{t,i}$ → **Softmax weights** $\alpha_{t,i}$ → **Context vector** $c_t$

Encoder provides hidden states $h_1, h_2, \ldots, h_n$. The decoder state $s_t$ is the query.

**Key distinction from Bahdanau: Luong scores using the current decoder state $s_t$.**

# Luong Attention: Alignment Score Calculation

Compute scores first, normalize with softmax, then form the context vector

## Three scoring functions

**Dot**

$$e_{t,i} = s_t^T h_i$$

Direct dot product; dimensions must match

**General**

$$e_{t,i} = s_t^T W_a h_i$$

Trainable matrix learns the comparison

**Concat**

$$e_{t,i} = v_a^T \tanh(W_a[s_t; h_i])$$

Trainable nonlinear scoring function

**Attention weights:** $\alpha_{t,i} = \exp(e_{t,i}) / \Sigma_j \exp(e_{t,j})$

**Context vector:** $c_t = \Sigma_i \alpha_{t,i} h_i$

# Luong Attention: Worked Score Example

Dot-product scoring for one decoder step

## Decoder state: $s_t = [1, 0]$

Encoder states: $h_1 = [1, 0]$  $h_2 = [0, 1]$  $h_3 = [1, 1]$

### Step 1  Compute dot-product alignment scores

$e_{t,1} = [1,0] \cdot [1,0] = 1$  $e_{t,2} = [1,0] \cdot [0,1] = 0$  $e_{t,3} = [1,0] \cdot [1,1] = 1$

**Score vector  $e_t = [1, 0, 1]$**

### Step 2  Apply softmax to the scores

$\alpha_t = \text{softmax}([1, 0, 1]) \approx [0.422, 0.155, 0.422]$

### Step 3  Compute context as the weighted sum

$c_t = 0.422h_1 + 0.155h_2 + 0.422h_3 \approx [0.844, 0.577]$

PART III

# Self-Attention

Tokens retrieve information from tokens in the same sequence.

# Why self-attention? Sequence context changes word meaning

**“The animal did not cross the street because it was tired.”**




The representation of “it” can retrieve evidence from “animal.”

Unlike recurrent propagation, one self-attention layer has direct token-to-token interactions.

# The three roles: Query, Key, Value

**Query (Q)**

What information is this position seeking?

**Key (K)**

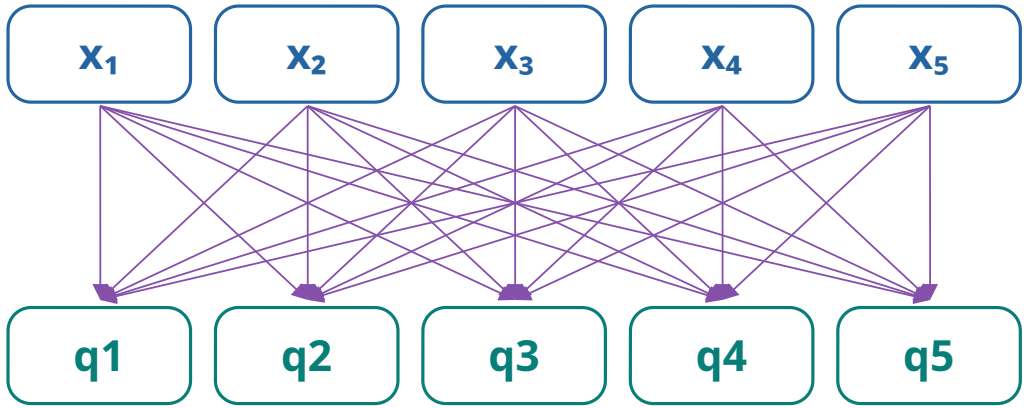
How should another position be matched?

**Value (V)**

What information is returned when selected?

$$Q = XW_Q \qquad K = XW_K \qquad V = XW_V$$

# Self-attention: all Q, K, V derive from one input X

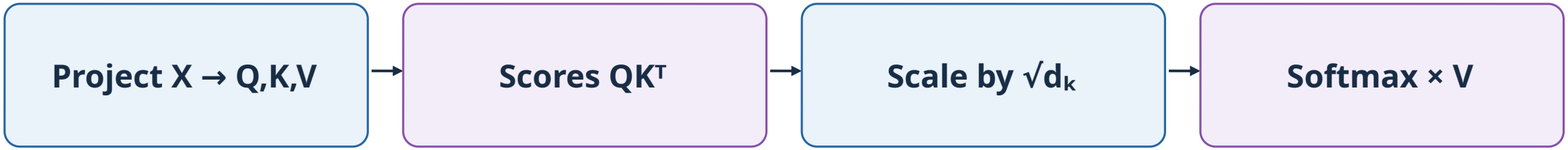



**Source**

The same input sequence X provides queries, keys, values.

$$Q = XWQ, \quad K = XWK, \quad V = XWV$$

# Scaled dot-product attention: all steps




$$\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d_k})\, V$$

The softmax is row-wise: each query has its own distribution across keys.

# Dimensions: shapes make the computation concrete

| Tensor | Shape | Meaning |
|---|---|---|
| X | n × dmodel | n token representations |
| Q | n × dk | Queries after WQ |
| K | n × dk | Keys after WK |
| V | n × dv | Values after WV |
| $QK^T$ | n × n | One score for every query–key pair |
| Attention output | n × dv | One context vector per query |

Example: n=4 tokens, dk=2, dv=3 → attention weights 4×4, output 4×3.

# Worked example 1: query–key similarity

$$q = [1, 0] \quad k_1 = [1,0] \quad k_2 = [0,1] \quad k_3 = [1,1]$$

$$qK^T = [1, 0, 1]$$

$$\text{scaled scores} = [1, 0, 1] / \sqrt{2}$$

**The query matches keys 1 and 3 equally under this example.**

# Worked example 2: softmax and values

$$\text{softmax}([0.707, 0, 0.707]) \approx [0.401, 0.198, 0.401]$$

$$v_1=[1,0],\ \ v_2=[0,2],\ \ v_3=[2,1]$$

$$\text{output} = 0.401v_1 + 0.198v_2 + 0.401v_3$$

$$\approx [1.203, 0.797]$$

Values, not keys, are averaged to form the output.

# Why divide by $\sqrt{d_k}$?

For unscaled dot products, score variance can increase with vector dimensionality.

$$\text{scores} = QK^T / \sqrt{d_k}$$

### Without scaling

Large-magnitude scores can push softmax toward sharp distributions, leading to small gradients.

### With scaling

Rescales scores to reduce the typical growth in dot-product magnitude.

Motivation assumes approximately independent, zero-mean query/key components with controlled variance.

# Bidirectional versus causal self-attention

| | Keys / source positions | | | |
|---|---|---|---|---|
| $q_1$ | 0.25 | 0.25 | 0.25 | 0.25 |
| $q_2$ | 0.10 | 0.30 | 0.20 | 0.40 |
| $q_3$ | 0.20 | 0.15 | 0.50 | 0.15 |
| $q_4$ | 0.40 | 0.10 | 0.10 | 0.40 |

**Bidirectional (encoder-style)**

| | Keys / source positions | | | |
|---|---|---|---|---|
| $q_1$ | **1.00** | 0.00 | 0.00 | 0.00 |
| $q_2$ | 0.40 | **0.60** | 0.00 | 0.00 |
| $q_3$ | 0.20 | 0.30 | 0.50 | 0.00 |
| $q_4$ | 0.10 | 0.20 | 0.30 | 0.40 |

**Causal (autoregressive decoder)**

Causal masking prevents each position from using future tokens.

# Masking must occur before softmax

$$A = \text{softmax}((QK^T / \sqrt{d_k}) + M)$$

**Allowed positions**

Mask contribution $M_{ij} = 0$

**Blocked positions**

Mask contribution $M_{ij} = -\infty$ (or sufficiently negative)

Padding masks suppress padding; causal masks suppress future token positions.

PART IV

# Cross-Attention

Queries come from the target side, while keys and values come from a separate source.

# Cross-attention: two sequences, different roles

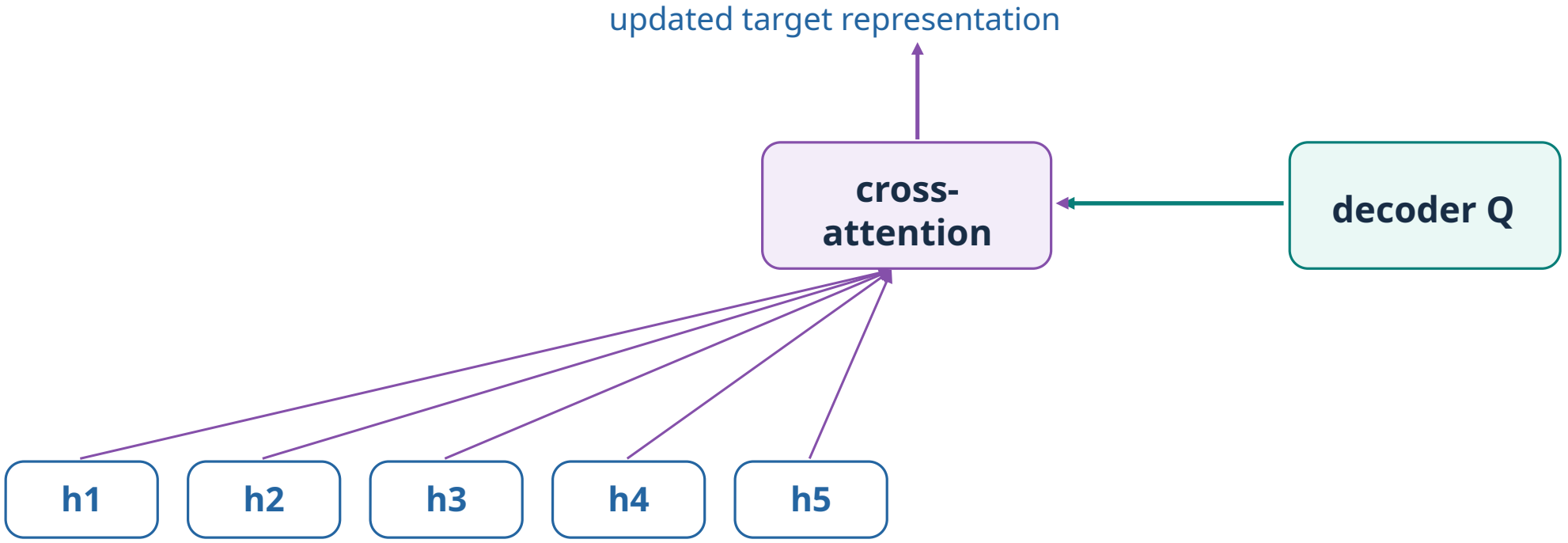



**Encoder representations → K,V**

**Decoder representations → Q**

**CrossAttention(Qtarget, Ksource, Vsource)**

# Cross-attention architecture: source retrieval




In a Transformer, cross-attention uses encoder outputs as K,V and decoder states as Q.

# Cross-attention dimensions and matrix multiplication

$$Q \in \mathbb{R}^{m \times d_k},\quad K \in \mathbb{R}^{n \times d_k},\quad V \in \mathbb{R}^{n \times d_v}$$

$$QK^T \in \mathbb{R}^{m \times n} \quad \rightarrow \quad \text{output} \in \mathbb{R}^{m \times d_v}$$

**m: target-side length**

One query for each target position.

**n: source-side length**

One key/value for each source position.

**Target and source lengths do not have to be equal.**

# Self versus cross attention: dependency map

| Property | Self-attention | Cross-attention |
| --- | --- | --- |
| Q source | Current sequence | Target/receiving sequence |
| K,V source | Same sequence | Another source sequence |
| Score shape | $n \times n$ | $m \times n$ |
| Common location | Transformer encoder; decoder masked self | Encoder–decoder bridge |
| Question asked | Which tokens here inform this token? | Which source tokens inform this target? |

Both can use scaled dot-product scoring and multiple heads.

PART V

# Multi-Head Attention

Multiple attention mechanisms run in parallel over different learned projections.

# Why multiple attention heads?

**Head 1**

May emphasize local syntactic relationships.

**Head 2**

May emphasize a longer-distance reference.

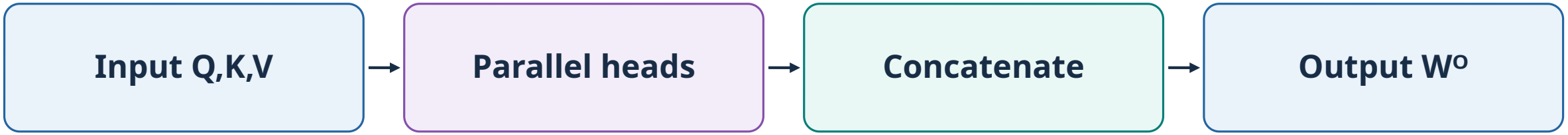
**Head 3**

May emphasize other token relationships.

$$\text{head}_j = \text{Attention}(QWQ_j, KWK_j, VWV_j)$$

Heads are learned; these possible functions are illustrations, not assigned roles.

# Multi-head computation: project, attend, concatenate




$$\text{MultiHead}(Q,K,V) = \text{Concat}(\text{head}_1,\ldots,\text{head}_h)\, W^O$$

Each head owns its own trainable Q, K, V projections.

# Multi-head: dimension bookkeeping

| Component | Example shape | Explanation |
|---|---|---|
| Model width dmodel | 512 | Input token representation |
| Number of heads h | 8 | Parallel heads |
| Each head dk = dv | 64 | 512 / 8 |
| One head output | n × 64 | One vector per query token |
| Concatenated output | n × 512 | 8 × 64 |
| Final $W^O$ | 512 × 512 | Mix the concatenated heads |

This is a standard illustrative Transformer configuration, not a universal requirement.

# Single head versus multi-head: what changes?

### Single head

One score matrix, one pattern of weighted retrieval.

### Multi-head

Several projected score matrices and value mixtures, later recombined.

$$\text{head}_j = \text{softmax}(Q_j K_j^T / \sqrt{dk})\, V_j$$

More heads do not guarantee that each learns a distinct interpretable feature.

# Where does attention appear in a Transformer?

### Encoder block

Unmasked self-attention
Q,K,V from source

### Decoder block (1)

Masked self-attention
Q,K,V from target prefix

### Decoder block (2)

Cross-attention
Q from decoder; K,V encoder

**Each attention sublayer is commonly multi-head.**

# Complexity: costs and tradeoffs

| Attention scope | Score matrix size | Implication |
| --- | --- | --- |
| Full self-attention | $n \times n$ | Quadratic in sequence length for score storage |
| Cross-attention | $m \times n$ | Depends on both source and target lengths |
| Local attention window | $n \times w$ (conceptually) | Fewer token-pair scores when $w \ll n$ |
| Multi-head (fixed total width) | $h$ separate head matrices | Adds distinct projections and per-head patterns |

Actual memory depends on masking, kernel implementation, batching and training/inference setup.

# Full comparison: all five requested mechanisms

| Mechanism | Scoring / retrieval | Typical Q / K / V |
|---|---|---|
| Bahdanau | Learned additive score | Decoder history / encoder states |
| Luong | Dot, general, concat | Current decoder state / encoder states |
| Self-attention | Typically scaled dot product | Same sequence / same sequence |
| Cross-attention | Typically scaled dot product | Target / separate source |
| Multi-head | Multiple projected attentions | Can be self or cross |

**These labels are not exclusive categories: e.g., multi-head cross-attention.**

# Classroom checkpoint: test understanding

- What differs between Bahdanau and Luong queries in their original papers?
- Which matrix has shape m × n in cross-attention?
- For scores [0,0,0], what does softmax return?
- Why is $\sqrt{d_k}$ used in scaled dot-product attention?
- Does multi-head automatically mean self-attention?
- In causal self-attention, can token 3 attend to token 4?

# Checkpoint solutions

- Bahdanau scores with the previous state; Luong global uses the current state.
- The query–key score matrix $QK^T$ has shape $m \times n$.
- The distribution is uniform: [1/3, 1/3, 1/3].
- To limit typical dot-product magnitude and softmax saturation.
- No: multi-head can operate as self- or cross-attention.
- No, token 4 is a future position and is masked.

# Key misconceptions to avoid

- “Bahdanau, self and multi-head are three mutually exclusive choices.” False.
- “Self-attention always sees future tokens.” False: causal masks restrict it.
- “Keys are averaged to produce the context.” False: values are averaged.
- “Attention weights prove a causal explanation.” Not in general.
- “Multi-head means the same matrix is reused in every head.” False.

# Research references and further reading


- Bahdanau, Cho & Bengio. Neural Machine Translation by Jointly Learning to Align and Translate. ICLR (2015).
- Luong, Pham & Manning. Effective Approaches to Attention-based Neural Machine Translation. EMNLP (2015).
- Vaswani et al. Attention Is All You Need. NeurIPS (2017).
- IIT Madras / AI4Bharat: CS6910 lecture on multi-headed self- and cross-attention.
- Prior class: IIT Bhilai lecture “From LSTM and GRU to Encoder–Decoder and Attention” (46 slides).

# One idea to remember

Attention = learned relevance weights applied to values.
Self/cross says WHERE; additive/dot says HOW; multi-head says HOW MANY.

---

End of lecture | Questions and numerical exercises